Major Course Output 3
CatBot

Release Date: October 24, 2025
Due Date: November 28, 2025 (Friday), 11:59 PM

1 Project Description

In this project, students will apply the concepts and theories they have learned in the course to develop a simple cat-catching bot that can adapt to the behaviors of different cats. Through this project, students should be able to demonstrate the following learning outcomes:

- **LO3.** Collaboratively design and implement machine learning models for supervised and unsupervised tasks
- **LO5.** Articulate ideas and present results in correct technical written and oral English.

2 Project Specifications

This section contains the specifications for this major course output.

2.1 Overview

The cats in campus are on the loose! DLSU PUSA has enlisted CCS students to help catch them. Of course, we can't expect CCS students to go running around the campus just to chase some cats. Instead, your group decides to build a bot that can do the job. In this project, you will implement the learning component of a bot called CatBot. CatBot is a bot that can catch cats in a 2-D grid. The bot must use reinforcement learning to automatically learn the behavior of different cats, and adapt its strategy accordingly.

2.2 The World

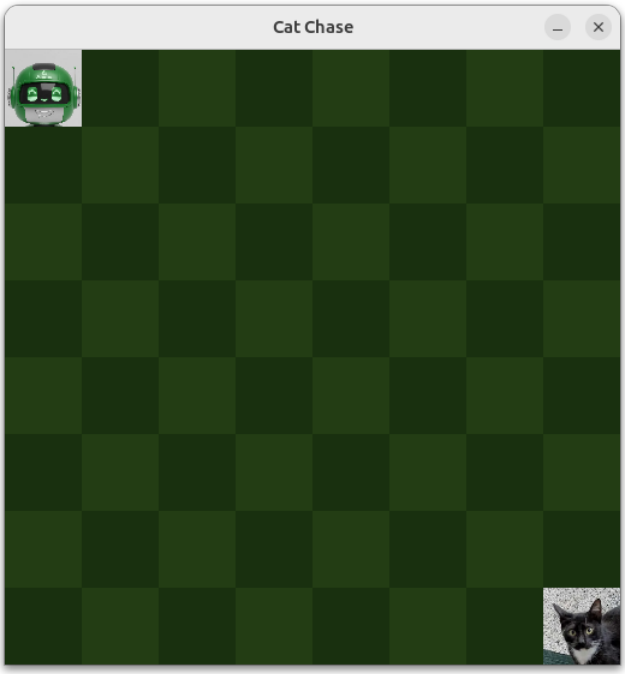

Figure 1: Screenshot of the world

The world is an 8×8 grid. In a given scenario, CatBot and a single cat will start at predefined cells in the grid. CatBot has five possible actions: move up, move down, move left, move right, or stay in place. The goal of CatBot is to catch the cat by ending up in the same cell as the cat. Everytime CatBot makes an action, the cat will also have a chance to perform one action. Different cats will behave differently according to their personality.

2.3 Catching Cats

You might think that catching the cats is just about following the shortest path to the cat. However, this is not always the case. Some cats are very playful, and will always run away if you just go straight for them. Therefore, a shortest-path strategy will not always work. Effective cat-catching requires learning the personality and behavior of the cat, knowing how to appease them until you can catch them. Therefore, CatBot must be implemeneted with a reinforcement learning component that can explore interactions with a given cat, learning how it responds to different actions, and then adapting its strategy accordingly.

2.4 The Cats

There will be ten different cats to catch. Each cat has a different personality and behavior. Your bot must use the same learning algorithm to adapt its behavior to any type of cat. In this project, you will be provided the specifications of the first five cats so you can experiment with your learning algorithm. The remaining five cats will be hidden and will be used for testing your bot during evaluation. CatBot must be capable of automatically learning the behavior of these hidden cats, even if you don't access to their specifications during development.

The following table shows the list of cats:

Cat Name	**Photo**	**Personality**	**Behavior Description**
Batmeow (`batmeow`)		Sleepy	Batmeow is too sleepy to move and just stays in place the entire time.
Mittens (`mittens`)		Energetic	Mittens had too much sugar and just moves around randomly.
Paotsin (`paotsin`)		Playful	The more you chase Paotsin, the more he runs away.
Squiddyboi (`squiddyboi`)		Athletic	Squiddyboi can jump over you, and will not hesitate to do so when threatened.
Peekaboo (`peekaboo`)		Ninja	Peekaboo can teleport to any edge of the map, but some say he has a weak side...
Spidercat (`spidercat`)		Stalker	*The behavior will remain a mystery until your project has been submitted.*
Cheddar (`cheddar`)		Intelligent	*The behavior will remain a mystery until your project has been submitted.*
Pumpkin Pie (`pumpkinpie`)		Angry	*The behavior will remain a mystery until your project has been submitted.*
Milky (`milky`)		Magical	*The behavior will remain a mystery until your project has been submitted.*
Taro (`taro`)		Enzo	*The behavior will remain a mystery until your project has been submitted.*
Trainer Cat (`trainer`)		N/A	This is a special robot cat. You can program its behavior to help you test the effectiveness of your learning algorithm in custom scenarios.

3 Starter Code

You are provided starter codes for this project. You are expected to use **Python 3.12** for this project. Earlier versions of Python may not be compatible with the OpenAI Gymnasium package used in this project.

Following are descriptions of the different files in the package:

- `cat_env.py` contains the implementation of the custom CatBot environment using the OpenAI Gymnasium framework. You are `NOT` allowed to modify this file in any way, except for the implementation of the Trainer cat, which you can modify to test your learning algorithm.
- `utility.py` contains helper functions for running the custom CatBot environment. You are also `NOT` allowed to modify this file in any way.
- `play.py` and `bot.py` are scripts for running CatBot in freeplay mode and bot mode, respectively. You are also `NOT` allowed to modify these files in any way.
- `training.py` contains the skeleton code for the reinforcement learning algorithm of CatBot. You are expected to implement the learning algorithm in this file.

It is recommended that you familiarize yourself first with the environment by running the `play.py` script. This runs the game in freeplay mode, where you can manually control CatBot using the keyboard. By default, the script loads the Batmeow cat scenario. You can change the cat type by specifying a parameter when running the script. For example, to run the Mittens cat scenario, you can run the following command in your terminal:

```
python play.py --cat mittens
```

Try to play around with the different cat types manually to understand their behaviors.

If you run the `bot.py` script, it will run the reinforcement learning training algorithm by calling the `train_bot` function in `training.py`. After training, the bot will automatically play the game using the learned optimal policy, and you will be able to see the bot in action. If the bot was trained properly, it should be able to catch the cat in a reasonable amount of time. You can also specify the cat type when running the script, similar to the `play.py` script. For example, to train and run the bot on the Paotsin cat scenario, you can run the following command in your terminal:

```
python bot.py --cat paotsin
```

You can also provide an optional parameter `--render` to visualize the behavior of the bot during different phases in the training. For example, to visualize the bot's behavior every 100 episodes during training, you can run the following command:

```
python bot.py --cat paotsin --render 100
```

By default, the `train_bot` function does not have any training logic, which means that the bot will not learn anything. Your primary task in this project is to implement a Q-learning algorithm by filling in the missing sections in `training.py` so that the bot can effectively learn to catch different types of cats.

4 Implementing the Reinforcement Learning Algorithm

You are expected to implement a reinforcement learning algorithm in the `training.py` file. For this project, Q-learning is enough, but you are free to explore other algorithms if you wish.

4.1 States and Actions Representation

The CatBot environment is an OpenAI Gymnasium environment. It is highly recommended that you follow online tutorials on OpenAI Gymnasium to familiarize yourself with how to interact with an environment, and how to implement reinforcement learning algorithms using it. Here are some suggestions on tutorials you can follow:

- OpenAI Gymnasium - Training an Agent Tutorial
- Q-Learning for Beginners

The states and actions are represented as follows:

- There are a total of 10,000 possible states (some of which are unused). Each state is represented as an integer from 0 to 9999, where the first two digits represent the row and column of CatBot, and the last two digits represent the row and column of the cat. For example, state 2305 means that CatBot is at row 2, column 3, while the cat is at row 0,

column 5. The rows and columns are 0-indexed. Because the grid is only 8×8, some states are never used, such as state 8800 (CatBot at row 8, column 8, cat at row 0, column 0).

- The actions are represented as integers from 0 to 4, where 0 represents moving up, 1 represents moving down, 2 represents moving left, 3 represents moving right, and 4 represents staying in place.
- By default, the CatBot environment **does not give out any rewards**. That is, the reward for doing any kind of action from any kind of state is 0. You are expected to design your own reward structure to effectively train CatBot to catch the cats. Note that you are **NOT** supposed to modify the `cat_env.py` file to implement the reward structure. Instead, you should implement the reward mechanism manually in the `training.py` file after receiving the state and action results from the environment.

4.2 Q-Learning

You are expected to implement the Q-learning algorithm in the `train_bot` function in `training.py`. You can follow online tutorials on how to implement Q-learning using OpenAI Gymnasium, but you must adapt it to the CatBot task. In general, each episode of training should cover the following:

1. Reset the environment to start a new episode.
2. Decide whether to explore or exploit.
3. Take the action and observe the next state.
4. Since this environment doesn't give rewards, compute reward manually.
5. Update the Q-table accordingly based on agent's rewards.

You are free to design your own reward structure, exploration strategy, hyperparameters, and any other aspects of the learning algorithm. However, there are some important restrictions:

- You are **NOT** allowed to modify any part of the code outside the designated areas.
- The training is fixed to 5000 episodes. You cannot change this.
- The entire training process, when executed, must be complete in at most 20 seconds (this does not include the time playing the animations, if `render` option is used).

You can test your learning algorithm by runnning the `bot.py` script on the different cat scenarios. Your goal is to make CatBot catch each cat in as few steps as possible after training.

Additionally, you can implement your own custom cat behavior by modifying the Trainer cat in `cat_env.py` to test the adaptability of your learning algorithm. Note that you are **NOT** allowed to modify the behavior of the other cats, or any other parts of the `cat_env.py` file.

5 Evaluation

To get perfect credit for the functionality, your bot must be able to catch **all ten cats**. Furthermore, in each scenario, your bot will only have a maximum of 60 moves to catch the cat. This restriction is in place to make sure that your bot is truly capable of learning the cat behavior and strategies, and not just blindly trying out actions until it gets lucky. If the bot exceeds 60 moves in a given scenario, the cat will be considered uncaught.

6 Report

In addition to the CatBot codes, you are required to write a report documenting the implementation of the project. The report should contain the following:

1. **Reinforcement Learning Algorithm**: A section describing the implementation of the Q-Learning algorithm, including the reward structure, exploration strategy, hyperparameters, and any other relevant details. You should also justify your design choices and explain how they contribute to the effectiveness of the learning algorithm.
2. **Evaluation and Performance**: A section containing an empirical evaluation of the performance of your bot on the different cat scenarios. You should provide quantitative results, such as the average number of steps taken to catch each cat after training, as well as qualitative observations on the behavior of the bot. You can also include graphs or tables to illustrate your results.

3. **Challenges**: A discussion on the difficulties encountered in developing the bot. What are the challenges that make it difficult for algorithms to adapat to new behavior?
4. **Table of Contributions:** A table showing the contributions of each member to the project.

The report should **not** exceed four pages with A-4 sized paper. If there is a strong need to have more pages, the group must ask permission from the instructor and justify it. There is no restriction on the formatting, as long as it is readable. Please be concise in writing the report and avoid repeating already-known definitinons. The bulk of the report should contain the group's personal ideas, insights, and implementations. Minimize the reiteration of already-known definitions. **You are not allowed to use generative AI for the report, even for writing improvements**. The report should be an accurate reflection of the group's personal understanding and engagement with the project.

7 Deliverables

There are two deliverables for this project: a zip file containing the source files for the project, and a pdf file containing the report. The zip file should contain all the source files needed to run the bot (including `training.py` which contains the `train_bot` function).

8 Academic Honesty

Honesty policy applies. Please take note that you are **NOT** allowed to borrow and/or copy-and-paste in full or in part any existing related program code from the internet or other sources (such as printed materials like books, or source codes by other people that are not online). **Violating this policy is a serious offense in De La Salle University and will result in a grade of 0.0 for the whole course**.

Please remember that the point of this project is for all members to learn something and increase their appreciation of the concepts covered in class. Each member is expected to be able to explain the different aspects of their submitted work, whether part of their contributions or not. **Failure to do this will be interpreted as a failure of the learning goals, and will result in a grade of 0 for that member**.

9 Credits

Some images used in this project were obtained from DLSU PUSA's cat directory. The detailed list of credits can be found in the `credits.txt` file included in the package. Images of the cats are credited to their respective photographers as indicated in the directory, and their use in this project falls under fair use. Please do not redistribute any part of this project publicly or commercially.

10 Generative AI Use Declaration

Some aspects of this project were prepared with the help of Gemini 2.5 and Claude Sonnet 3.5, namely:

- Assistance in preparing the starter code
- Generating images for the CatBot

The idea and design of the project is the original idea of the instructor, and generative AI was simply used to make its production more efficient. Any code or images generated by generative AI has been thoroughly reviewed, modified, and synthesized to align with the instructor's intentions and objectives for this project. The instructor takes full accountability for all the contents of this package.

Please note that students are **NOT** allowed to use generative AI for any code generation or text generation in this project.